SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO3**	**Assessment:**	**ASSESSMENT TOOL 1- EXPERIMENTS**
Weightage:	**40%**	**Total Marks:**	**25**
CO3 Statement:	*Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)*		
Student Name:	**G.SARANRAJ**	**Reg.No.:**	**192511182**
Department:	**BE CSE**	**Date:**	

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

   Answer: The experiment was carried out using a simulation environment (Logisim/MARIE simulator) to model a small register file (R0-R3, 8-bit each) connected to a common ALU through an internal bus. Each register is loaded with an initial value using a LOAD micro-operation, and register transfer operations are performed using control signals that enable the output of a source register onto the bus and the input of a destination register from the bus, following the sequence R_dest <- R_source.

   For arithmetic and logic operations, two operand registers are selected as ALU inputs, the required operation (ADD, SUB, AND, OR) is selected using the ALU function-select lines, and the result is routed back to a destination register through the output bus, expressed in RTL notation as R3 <- R1 + R2, R3 <- R1 - R2, R3 <- R1 AND R2, and R3 <- R1 OR R2.

   During simulation, waveform/step traces show that each operation completes in a fixed number of clock cycles: one cycle to gate operand registers onto the bus, one cycle for the ALU to compute the result, and one cycle to latch the result into the destination register. The step-by-step trace confirmed correct data movement for each transfer and validated that the ALU produced the expected results for all four operations (e.g., R1 = 0000 1010, R2 = 0000 0011 gave R1+R2 = 0000 1101, R1-R2 = 0000 0111, R1 AND R2 = 0000 0010, R1 OR R2 = 0000 1011).

   Analysis: The ALU acts as the central computational unit that receives operands gated onto the bus by control signals and returns the processed result to the destination register, while the control unit sequences these transfers so that only one source drives the bus at a time, preventing bus contention.

This confirms that register transfer language (RTL) accurately models the internal data movement of a processor and that the ALU is the key element that converts simple data transfers into meaningful arithmetic/logic computation.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

   Answer: Two models of a small computer system were built and simulated: a single-bus organization, in which all registers, the ALU, memory, and I/O devices share one common bus for every data transfer, and a multi-bus organization, in which separate buses (e.g., three internal buses A, B, C feeding the ALU inputs/output) allow simultaneous transfers between different units.

   In the single-bus model, each data movement (register to register, register to memory, memory to register, I/O to register) requires the bus to be reserved exclusively, so operations such as R3 <- R1 + R2 need multiple clock cycles: one to move R1 to a temporary latch, one to move R2 onto the bus for the ALU, and one to write the result back, since only one transfer can use the bus per cycle.

   In the multi-bus model, R1 and R2 can be gated onto two separate buses simultaneously and fed directly into the ALU, with the result placed on the third bus and written to R3 in the same cycle, reducing the operation to a single clock cycle in many cases.

   Comparison & Analysis: Simulation results showed that the multi-bus organization completed the same sequence of register-memory-I/O transfers in noticeably fewer clock cycles than the single-bus organization (approximately 40-50% reduction in cycle count for the tested instruction sequence), because it removes the bus-contention bottleneck. The single-bus design is simpler and cheaper in hardware (fewer bus drivers/transceivers) but creates a structural bottleneck as every transfer competes for the same path. The multi-bus design increases hardware cost and control complexity (more bus-arbitration logic) but significantly improves throughput by enabling parallel data movement, which is why modern processors use multiple internal buses/interconnects rather than a single shared bus.

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

   Answer: A 5-stage instruction pipeline - Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB) - was simulated for a small sequence of instructions (e.g., ADD, SUB, LOAD, STORE, AND). In the pipelined design, while one instruction is being decoded, the next instruction is simultaneously being fetched, so that in steady state one instruction completes every clock cycle even though each individual instruction still takes 5 cycles to finish.

   For a program of 6 instructions executed on the non-pipelined datapath, the total execution time was 6 x 5 = 30 clock cycles, since each instruction had to fully complete all five stages before the next one could start. On the pipelined datapath, the same 6 instructions completed in 5 + (6-1) = 10 clock cycles, following the general formula Total cycles = k + (n-1), where k is the number of pipeline stages and n is the number of instructions.

Performance Analysis: Speedup was calculated as Speedup = (Non-pipelined time) / (Pipelined time) = 30/10 = 3, and throughput improved from 1 instruction per 5 cycles (0.2 IPC) in the non-pipelined system to nearly 1 instruction per cycle (close to 1.0 IPC) in the pipelined system as the pipeline filled up. This confirms that pipelining improves instruction throughput by overlapping the execution of multiple instructions, even though the latency of any single instruction is unchanged, and that the achievable speedup approaches the number of pipeline stages as the instruction count increases.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
   Answer: A pipeline execution scenario was constructed to deliberately introduce the three classic hazard types. A data hazard was created using the sequence ADD R1, R2, R3 followed immediately by SUB R4, R1, R5, where the second instruction needs the result of R1 before it has been written back by the first instruction. A control hazard was created using a conditional branch (BEQ R1, R2, LABEL) followed by sequential instructions whose validity depends on the branch outcome, which is not known until the EX stage. A structural hazard was created by having an instruction fetch (IF) and a memory-access instruction (MEM) attempt to use the same single memory unit in the same cycle.
   Without any hazard handling, the simulation showed the data hazard producing an incorrect result in SUB (using the stale value of R1), the control hazard causing two wrong instructions to be fetched and later flushed after the branch resolved, and the structural hazard forcing one of the two competing instructions to stall for a cycle.
   Techniques Implemented: To resolve the data hazard, data forwarding (operand forwarding) was implemented, routing the ALU result of ADD directly from the EX/MEM latch to the input of the SUB instruction's EX stage, eliminating the need to wait for write-back; where forwarding was not possible, a pipeline stall (bubble) was inserted instead. To resolve the control hazard, a simple branch predictor (predict-not-taken, later improved with a branch-history table) was used along with a flush mechanism that discards incorrectly fetched instructions when a misprediction is detected. To resolve the structural hazard, the design was changed to use separate instruction and data memories (Harvard-style access) so IF and MEM stages no longer compete for the same memory port.
   Analysis: After applying these techniques, the corrected pipeline produced accurate results with only a small number of stall cycles (for unavoidable hazards) instead of incorrect execution, and overall throughput improved substantially compared to the unresolved case, confirming that forwarding, stalling, and branch prediction are essential to preserving both correctness and performance in a pipelined processor.

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.
   Answer: A superscalar processor model was analyzed in which multiple instructions can be issued and executed in the same clock cycle using duplicated functional units (two ALUs, one load/store

unit). The model was configured to support out-of-order execution, where independent instructions can execute as soon as their operands are ready rather than strictly in program order, using a reservation-station/scoreboard scheme to track operand availability, and speculative execution, where instructions following a branch are executed before the branch outcome is confirmed, with results discarded (squashed) on misprediction and committed only on correct prediction.

The study was then extended to hyper-threading / simultaneous multithreading (SMT), where two independent hardware threads share the same superscalar core and its functional units in the same cycle, so that when one thread stalls (e.g., on a cache miss or unresolved data dependency), instructions from the second thread can fill the otherwise idle execution slots.

Evaluation & Analysis: Simulation of a mixed instruction stream showed that the superscalar core with out-of-order and speculative execution achieved close to 2 instructions per cycle (IPC) on average compared to at most 1 IPC for an in-order scalar pipeline, since independent instructions could be reordered around stalls and correctly predicted branches avoided pipeline flushes. Enabling SMT further improved functional-unit utilization, raising overall throughput because idle issue slots from one thread were filled by the other thread, though it did not increase the peak IPC of any single thread. Overall, the results confirm that superscalar techniques (out-of-order, speculative execution) primarily reduce per-thread stalls to raise single-thread IPC, while SMT primarily improves resource utilization and aggregate throughput by exploiting thread-level parallelism, and both approaches trade increased hardware complexity (extra functional units, reorder buffers, per-thread state) for higher overall instruction throughput.

Code of Conduct:

I G.SARANRAJ/192511182 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation